PROJECT REPORT

Vehicle Insurance Management System (VIMS)

A Comprehensive Solution for Digital Insurance Operations

Submitted By:
Gudipalli Srineel Reddy
March 9, 2026

Table of Contents

1. Abstract 1
2. Problem Statement 1
3. Project Objectives 1
4. Technology Stack & Core Engines 1
   4.1. Server-Side Technologies (Backend) 1
   4.2. Client-Side Technologies (Frontend) 1
5. System Architecture 1
   5.1. Interaction Flow Example (Raising a Claim) 1
6. Advanced Business Logic Engines 1
   6.1. Vehicle Risk Score Engine 1
   6.2. Automatic Premium Calculator Engine 1
   6.3. Claims Risk Engine (Adjudication Logic) 1
7. Detailed Module Explanations 1
   7.1. Admin Module 1
   7.2. Customer Module 1
   7.3. Agent Module 1
   7.4. Underwriter Module (Risk Assessment) 1
   7.5. Surveyor Module (Field Inspection) 1
   7.6. Claims Officer Module (Financial Adjudication) 1
8. Complete Business Workflow 1
   8.1. Policy Lifecycle: Registration to Activation 1
   8.2. Claim Processing Lifecycle: Incident to Closure 1
9. UML Diagrams 1
   9.1. Use Case Diagram 1
   9.2. Class Diagram 1
   9.3. Entity-Relationship (ER) Diagram 1
   9.4. Workflow / Activity Diagram 1
   9.5. Policy Sequence Diagram 1
10. Database Design 1
   10.1. Major Entities and Relationships 1
11. API Documentation Overview 1

11.1. Authentication APIs (JWT Engine) 1
11.2. Policy Management APIs 1
11.3. Claim Management APIs 1
12. Key Features & Technical Challenges 1
12.1. Key Features 1
12.2. Development Challenges & Solutions 1
13. Future Enhancements 1
14. Conclusion 1

1. Abstract

The Vehicle Insurance Management System (VIMS) is an advanced, centralized digital platform engineered to modernize and streamline the traditional, multi-faceted processes associated with vehicle insurance. Historically, the insurance sector has relied heavily on manual data entry, physical document handling, and localized data storage, leading to systemic inefficiencies, delayed claim processing, and frequent miscommunication among stakeholders.

VIMS digitalizes the entire insurance lifecycle — from initial policy inquiry and purchase through to claim submission, adjudication, and final settlement. Built on a robust ASP.NET Core 8 backend paired with a dynamic Angular 20+ Single Page Application, and underpinned by Microsoft SQL Server with Entity Framework Core Code-First migrations, the system delivers a seamless, role-driven experience for all four key actor groups: Customers, Agents/Underwriters, Claims Officers, and Administrators.

By automating premium calculations using a standardized Insured Declared Value (IDV) depreciation grid, implementing a multi-stage claim approval workflow, enforcing role-based access control at both the API and UI layers, and maintaining an immutable audit trail of every critical system action, VIMS delivers measurable improvements in transparency, processing speed, financial accuracy, and regulatory compliance for insurance providers and policyholders alike.

2. Problem Statement

The vehicle insurance industry in its traditional form relies heavily on manual, paper-based processes that introduce significant operational inefficiencies and customer experience failures. The following pain points directly motivate the development of VIMS:

- **Excessive Manual Processing & Paperwork:** A high reliance on physical documents for policy issuance and claim evidence collection leads to substantial administrative overhead, archiving costs, a high risk of document loss, and slow information retrieval times.
- **Delayed Claim Settlements:** The absence of integrated digital workflows means claims must be manually transitioned between customers, surveyors, and claims officers — introducing severe bottlenecks and weeks-long resolution delays.
- **Lack of Real-Time Transparency:** Customers have no real-time visibility into the status of their policy applications or active claims. This creates customer anxiety, drives high call volumes, and erodes trust in the insurance provider.
- **Error-Prone Premium Calculations:** Manual IDV and premium computations are susceptible to human error. Inconsistent application of depreciation tables leads to financial discrepancies, customer disputes, and potential regulatory exposure.
- **High Fraud Vulnerability:** Without structured digital audit trails and automated document verification workflows, identifying fraudulent or inflated claims is difficult, reactive, and expensive.
- **Absence of a Centralized Record System:** Policy, vehicle, payment, and claim data is scattered across spreadsheets, physical files, and disparate systems, making consolidated reporting and compliance auditing extremely difficult.

3. Project Objectives

The primary goals of the VIMS project are defined as follows:

1. **Automate Policy Operations:** Digitize the purchasing, tracking, renewal, cancellation, and transfer of insurance policies, including server-side algorithms for accurate premium calculation based on vehicle specifications and chosen plan parameters.
2. **Structured Claim Management:** Provide a fully digital portal for claim submission and adjudication with automated payout estimations, evidence upload, multi-stage review workflows, and enforcement of claim payout caps.
3. **Role-Based Workflow Enforcement:** Establish clear digital boundaries and responsibilities for Admins, Agents/Underwriters, Customers, Claims Officers, and Surveyors through RBAC enforced at both the API and Angular UI layers.
4. **Data Integrity & Security:** Protect sensitive credentials and financial records using password hashing, JWT-based stateless authentication, CAPTCHA validation, and HTTPS-enforced communication throughout.
5. **Immutable Audit Traceability:** Maintain a write-once AuditLog of every critical system action — who performed what operation, on which record, at what time — for compliance, dispute resolution, and fraud prevention.
6. **Scalable Architecture:** Design a modular, layered system capable of accommodating growing user volumes, new insurance product types, and future integrations with payment gateways and AI fraud detection services.

4. Technology Stack & Core Engines

VIMS is built on a carefully selected set of modern, enterprise-grade technologies. Each technology choice was made to maximize maintainability, developer productivity, and runtime scalability.

4.1. Server-Side Technologies (Backend)

Technology	Version / Detail	Role in VIMS
ASP.NET Core	8.0	Core Web API — routing, middleware pipeline, dependency injection, model validation
C#	12	Primary language — controllers, services, repositories, domain models
Entity Framework Core	Code-First	ORM — schema versioning via migrations, LINQ-based queries, relationship management
JWT Authentication	System.IdentityModel	Stateless auth — signed tokens issued on login, validated per request by middleware
Service-Repository Pattern	Custom	Services enforce business rules; Repositories abstract all EF Core data access
AuthService	Custom	Validates CAPTCHA, hashes passwords, issues JWT, manages security questions
PricingService	Custom	Calculates IDV, premium, invoice amounts server-side — client values are ignored
ClaimsService	Custom	Validates claim coverage, enforces payout caps, manages claim lifecycle transitions
PDF Generation Service	Custom	Generates downloadable payment invoices on demand
Swagger / OpenAPI	Swashbuckle 6.x	Auto-generated interactive API documentation at /swagger
Static File Server	wwwroot/uploads	Serves uploaded vehicle documents and claim evidence files

4.2. Client-Side Technologies (Frontend)

Technology	Version / Detail	Role in VIMS
Angular	20+	Core SPA framework — routing, modules, component lifecycle, reactive state
TypeScript	5.x	Strongly-typed JavaScript — maintainable service, component, and model code

Technology	Version / Detail	Role in VIMS
Tailwind CSS / Vanilla CSS	Latest	Utility-first styling — responsive, consistent UI across all role dashboards
RxJS	Latest	Reactive streams — async HTTP calls, event handling, real-time state updates
Angular HTTP Interceptors	Built-in	Automatic JWT injection on every outbound request; 401 → redirect to login
Angular Route Guards	CanActivate	Client-side RBAC — blocks unauthorized navigation by decoding JWT role claims

5. System Architecture

VIMS follows an N-Tier Architecture that enforces strict separation of concerns across five distinct layers. Each layer has a single well-defined responsibility and communicates only with the layer directly adjacent to it — preventing business logic from leaking into controllers or data access from appearing in services.

Layer	Component	Responsibility
Presentation	Angular SPA (Frontend)	Renders role-specific UI, manages JWT tokens in localStorage, enforces Route Guards
API Gateway	ASP.NET Core Controllers	Receives HTTP requests, validates JWT claims, delegates to services, returns JSON
Business Logic	Service Layer (C# Services)	Executes all business rules — pricing, coverage validation, status transitions, audit logging
Data Access	Repository Layer (EF Core)	Abstracts all SQL Server queries via repository interfaces and EF Core DbContext
Persistence	Microsoft SQL Server	Stores all relational data — users, vehicles, policies, claims, payments, audit logs
File Storage	wwwroot/uploads	Organized hierarchical storage for vehicle documents and claim evidence files
PDF Generation	Custom PDF Service	Produces downloadable invoice PDFs for policy payments and claim settlements

5.1. Interaction Flow Example (Raising a Claim)

The following numbered sequence demonstrates how a complete API request flows through all architecture layers:

7. **Customer action in Angular triggers an HTTP POST to /api/Customer/claim/submit.**
8. **Angular HTTP Interceptor automatically attaches the JWT Bearer token to the Authorization header.**
9. **ASP.NET Core middleware validates the JWT signature and extracts the CustomerId and Role claims.**
10. **CustomerController delegates to ClaimsService, passing the validated claim DTO.**
11. **ClaimsService validates the ClaimType against the PolicyPlan coverage flags (TheftCover, OwnDamageCover, etc.).**
12. **ClaimsService calls ClaimRepository.Add() — EF Core executes a parameterized INSERT against SQL Server.**

13. **AuditLogService writes an immutable entry (Action=ClaimSubmitted, EntityId=ClaimId) as the final step.**
14. **The Controller returns HTTP 201 Created with the new Claim record as JSON to the Angular frontend.**

6. Advanced Business Logic Engines

All financial and risk calculations in VIMS are performed exclusively server-side. Clients submit only vehicle details and plan selections — they never influence calculated prices. This design prevents price manipulation and ensures every calculation is auditable and reproducible.

6.1. Vehicle Risk Score Engine

Before the premium is calculated, the system computes a Vehicle Risk Score based on multiple vehicle attributes. This score modifies the Own Damage (OD) component of the premium.

Factor	Low Risk	High Risk
Fuel Type	Electric / CNG	Petrol / Diesel (higher fire risk)
Vehicle Type	Sedan / Hatchback	Sports / SUV (higher repair cost)
Vehicle Age	< 2 years (new)	5+ years (more wear, depreciation)
Cubic Capacity	< 1000cc	> 2000cc (higher TP premium band)
Usage	Private	Commercial (higher mileage, exposure)

6.2. Automatic Premium Calculator Engine

The PricingService uses a standardized IDV depreciation grid to calculate the Insured Declared Value (IDV) — the current market value of the vehicle — which forms the basis of the Own Damage premium component.

Vehicle Age	Depreciation Rate	IDV Calculation
Less than 6 months	5%	IDV = Ex-showroom Price × 0.95
6 months to 1 year	15%	IDV = Ex-showroom Price × 0.85
1 to 2 years	20%	IDV = Ex-showroom Price × 0.80
2 to 3 years	30%	IDV = Ex-showroom Price × 0.70
3 to 4 years	40%	IDV = Ex-showroom Price × 0.60
4 to 5 years	50%	IDV = Ex-showroom Price × 0.50
Over 5 years	60%	IDV = Ex-showroom Price × 0.40

Premium Formula — Third Party Only: Premium = TP Fixed Rate (government-regulated, by vehicle cubic capacity)

Premium Formula — Comprehensive: Premium = (IDV × OD Rate%) + TP Fixed Rate

Premium Formula — Zero Depreciation: Premium = Comprehensive Premium + Zero Dep Surcharge%

Discounts applied by PricingService: Multi-year commitment (up to 12% OD reduction for SelectedYears ≥ 2), Loyalty/Renewal discount for returning customers (IsRenewed = true), and No-Claim Bonus (NCB) stepped from 20% to 50% based on consecutive claim-free years.

6.3. Claims Risk Engine (Adjudication Logic)

ClaimsService applies a different payout formula for each ClaimType. The final payout is always capped at MaxCoverageAmount minus DeductibleAmount from the linked PolicyPlan — a hard limit enforced at the service layer before the officer can save a decision.

Claim Type	Payout Formula	Key Notes
Theft	Current IDV	IDV recalculated at claim time, not policy issuance time.
Own Damage (Standard)	Repair Cost − Part Depreciation − Deductible	Part depreciation rate applied to replaced parts only, not labour.
Own Damage (Zero Dep)	Repair Cost − Deductible	Zero Dep add-on eliminates part depreciation entirely.
Third Party	Assessed Liability − Deductible	Based on court-assessed or negotiated third-party liability amount.

7. Detailed Module Explanations

VIMS is structured around five user roles. Each has a dedicated Angular module with role-specific dashboards and corresponding restricted API endpoints. Route Guards on the frontend and Authorization attributes on controllers enforce these boundaries consistently.

7.1. Admin Module

Feature Area	Details
User Management	Create Agent and Claims Officer accounts. Activate or deactivate any user account. View all registered users with their roles and status.
Policy Plan Management	Create, activate, and deactivate PolicyPlans. Configure coverage flags (OwnDamage, Theft, ThirdParty), deductibles, maximum coverage amounts, add-ons, and base premiums.
System-Wide Oversight	View all VehicleApplications, Policies, Claims, and Payments across all users. Full visibility into the operational health of the entire system.
Immutable AuditLog Access	Read-only access to the complete AuditLog. Filter by date, category, user, or entity. Essential for compliance auditing and fraud investigation.

7.2. Customer Module

Feature Area	Details
Explore Plans	Browse all active PolicyPlans including coverage details, add-on availability, deductibles, and base premiums — accessible on the Explore Plans page without login.
Quote Calculator	Get instant premium estimates before committing to an application — PricingService calculates server-side from vehicle details and selected plan.
Vehicle Application	Submit a VehicleApplication with vehicle details (make, model, year, registration number, fuel type, vehicle type) and upload supporting documents (RC Book, ID Proof).
Policy Management	View all owned policies, track year-by-year status, pay annual premiums, renew policies, cancel active policies, or initiate policy transfers.
Claims	File claims against Active policies (OwnDamage, Theft, ThirdParty), upload evidence documents (minimum one required), and track claim status through adjudication.
Payment History	View all past payments and download PDF invoices for any completed transaction.

Feature Area	Details
Policy Transfer	Initiate outgoing transfers to other customers and accept or reject incoming transfer requests — both parties must actively consent.

7.3. Agent Module

Feature Area	Details
Application Queue	View all VehicleApplications assigned to this agent, with filtering by status (Pending, Approved, Rejected).
Document Review	Access and review all uploaded VehicleDocuments for each application before making an underwriting decision.
Approve / Reject	Approve applications — triggering automatic Vehicle and Policy creation by the backend — or reject with a mandatory written reason.
Customer Lookup	Search and view customer account details to assist in application verification.

7.4. Underwriter Module (Risk Assessment)

The Underwriter role within the Agent module provides an elevated level of scrutiny for high-value or complex vehicle applications. The underwriter reviews the Vehicle Risk Score computed by the Risk Engine, examines the applicant's claims history (ClaimCount on previous policies), verifies all submitted documents against government records where available, and has the authority to request additional documentation before a final approval decision is issued.

7.5. Surveyor Module (Field Inspection)

The Surveyor performs physical or digital vehicle inspection when a Claims Officer requires independent damage assessment — particularly for Own Damage claims with significant repair costs. The Surveyor produces an itemized inspection report with per-part repair cost estimates and photographic evidence. This report feeds directly into the OwnDamage payout formula used by the Claims Officer: Payout = Repair Cost − Part Depreciation − Deductible.

7.6. Claims Officer Module (Financial Adjudication)

Feature Area	Details
Claims Queue	View all submitted claims across all customers with status filtering (Pending, Under Review, Approved, Rejected, Closed).
Claim Detail Review	Access full claim details — policy info, vehicle info, customer details, ClaimType, and all uploaded ClaimDocuments (photos, FIR copy,

Feature Area	Details
	repair estimates, Surveyor report).
Payout Decision	Approve claims with a calculated ApprovedAmount (subject to MaxCoverageAmount − Deductible cap enforced by ClaimsService) or reject with a documented reason and DecisionType.
My Claims	View the officer's personal caseload — only claims where ClaimsOfficerId matches the logged-in officer.

8. Complete Business Workflow

8.1. Policy Lifecycle: Registration to Activation

The policy acquisition workflow takes a prospective customer from initial interest through to a fully activated insurance contract:

15. **Registration / Login:** Customer registers (AuthService validates CAPTCHA, hashes password, issues JWT) or logs into an existing account.
16. **Plan Discovery:** Customer browses all active PolicyPlans on the Explore Plans page — no login required. Plan type, coverage flags, deductibles, and base premiums are visible.
17. **Quote Calculation:** Customer enters vehicle details; PricingService returns an instant InvoiceAmount estimate. This is read-only and does not create any records.
18. **Application Submission:** Customer submits a VehicleApplication with vehicle details, selected PlanId, SelectedYears, and uploads at least one VehicleDocument. Status = Pending.
19. **Agent Assignment:** The system sets AssignedAgentId on the application. The agent sees it in their Pending Applications queue.
20. **Underwriting Review:** Agent examines vehicle details and all uploaded VehicleDocuments. Agent issues an Approve or Reject decision.
21. **Rejection Path:** If rejected, RejectionReason is recorded, application Status = Rejected, and the customer is notified.
22. **Policy Creation:** On approval, the system automatically creates a Vehicle record and a Policy record (Status = Pending Payment). PricingService sets PremiumAmount, IDV, InvoiceAmount, StartDate, EndDate, CurrentYearEndDate.
23. **Payment:** Customer pays the InvoiceAmount. A Payment record is created (Status = Completed) and IsCurrentYearPaid = true.
24. **Activation:** Policy Status transitions to Active. A unique PolicyNumber is generated. The policy is now fully operational.

8.2. Claim Processing Lifecycle: Incident to Closure

The claim settlement workflow covers the complete lifecycle from a customer filing a claim to final payout or rejection:

25. **Claim Submission:** Customer selects an Active policy, chooses ClaimType (Theft, OwnDamage, ThirdParty), provides incident details, and uploads supporting ClaimDocuments. ClaimsService validates coverage before creating the record. Status = Pending. ClaimCount on Policy +1.
26. **Officer Assignment:** Claim appears in all Claims Officers' queue. An officer self-assigns it (or auto-assignment to least-loaded officer). Status = Under Review. ClaimsOfficerId set.
27. **Document Review:** Officer examines the full claim — policy details, coverage flags, all uploaded ClaimDocuments, and optional Surveyor inspection report.

28. **Payout Decision:** Officer enters ApprovedAmount and DecisionType (Approve) or RejectionReason (Reject). ClaimsService enforces the cap: ApprovedAmount ≤ MaxCoverageAmount − DeductibleAmount.
29. **Settlement:** On approval, a Payment record is auto-generated for ApprovedAmount. Claim Status = Approved then Closed.
30. **Rejection:** RejectionReason and DecisionType recorded. Claim Status = Rejected. Customer is notified.

9. UML Diagrams

The following UML diagrams provide a comprehensive visual representation of the VIMS system architecture, domain model, and operational workflows. Each diagram captures a different perspective of the system — from high-level actor interactions to detailed entity relationships and time-ordered message sequences.

9.1. Use Case Diagram

The Use Case Diagram illustrates the interactions between the four primary system actors (Admin, Agent, Customer, Claims Officer) and the system's core functional use cases. Each actor is shown with the set of operations they are permitted to perform. Shared use cases (such as Login) show which actors share access to that function.

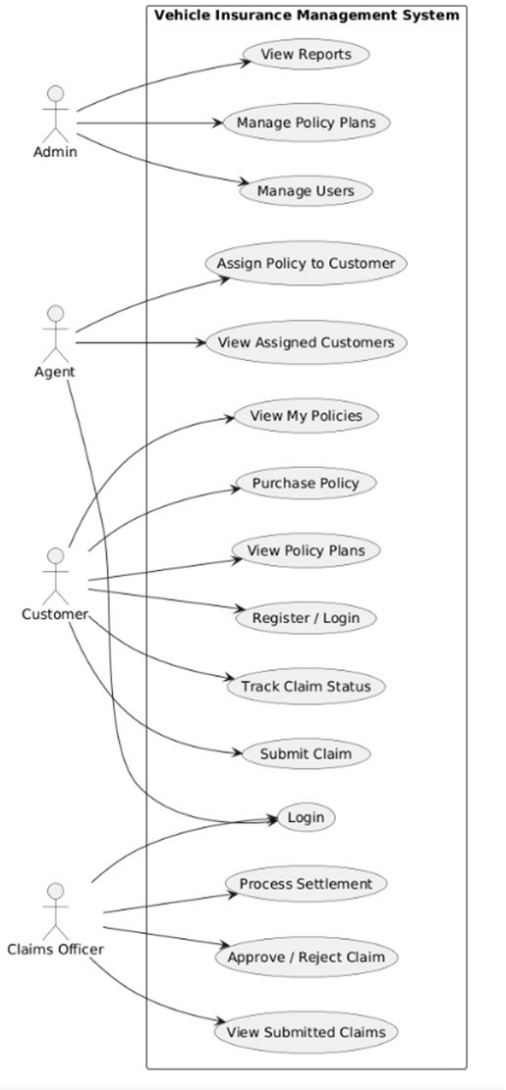

Figure 1: Use Case Diagram — Vehicle Insurance Management System

9.2. Class Diagram

The Class Diagram shows the core domain model of VIMS — the key classes, their attributes with data types, and the associations between them. It covers User, Vehicle, PolicyPlan, Policy, Claim, Payment, and ClaimDocument, along with the cardinalities and navigation directions of each relationship (owns, insured under, defines, generates, has, contains, files, handles).

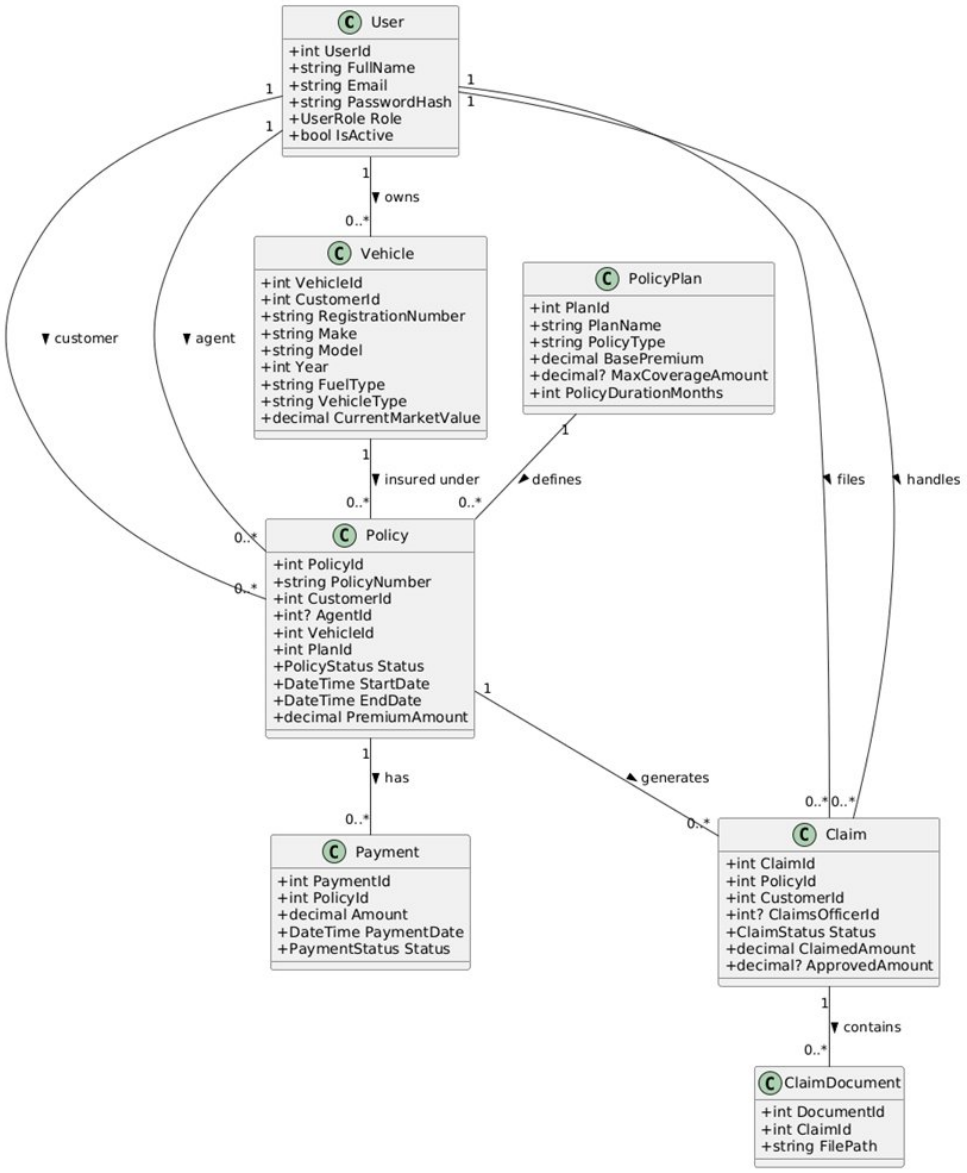

Figure 2: Class Diagram — VIMS Domain Model

9.3. Entity-Relationship (ER) Diagram

The Entity-Relationship Diagram provides a formal data-level view of the VehicleInsuranceDB schema as implemented in SQL Server via EF Core Code-First migrations. Every entity is shown with its primary key, foreign keys, and attribute types. Relationship lines indicate cardinality — the schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and enforce referential integrity.

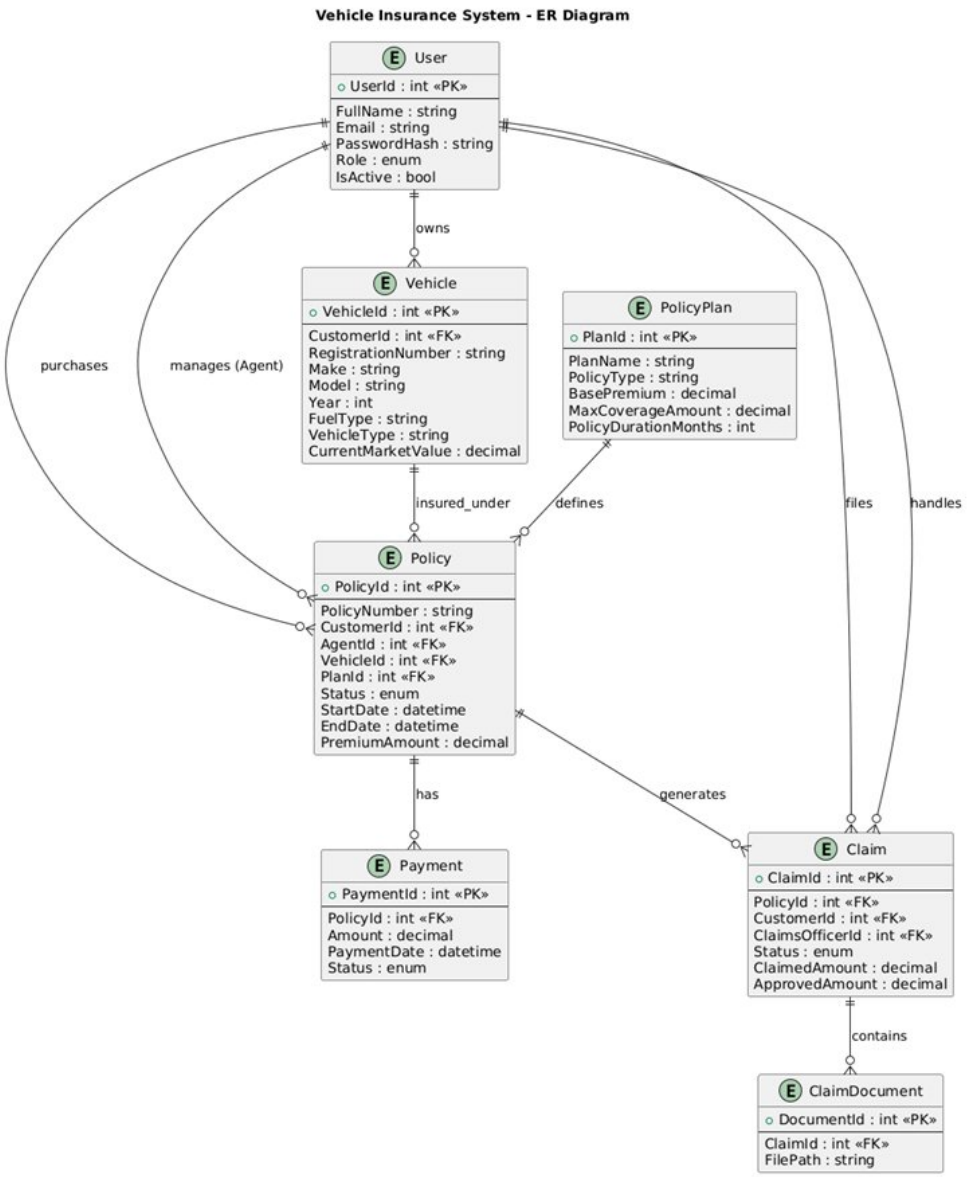

Figure 3: Entity-Relationship Diagram — VehicleInsuranceDB Schema

9.4. Workflow / Activity Diagram

The Activity Diagram captures the complete end-to-end business process flow of VIMS — from initial user registration through policy purchase, claim submission, and eventual policy expiry or renewal. It covers both the Customer and Agent/Claims Officer parallel activities, showing decision points (Role selection, Payment Successful, Documents Valid, Policy Duration Over) and the resulting state transitions.

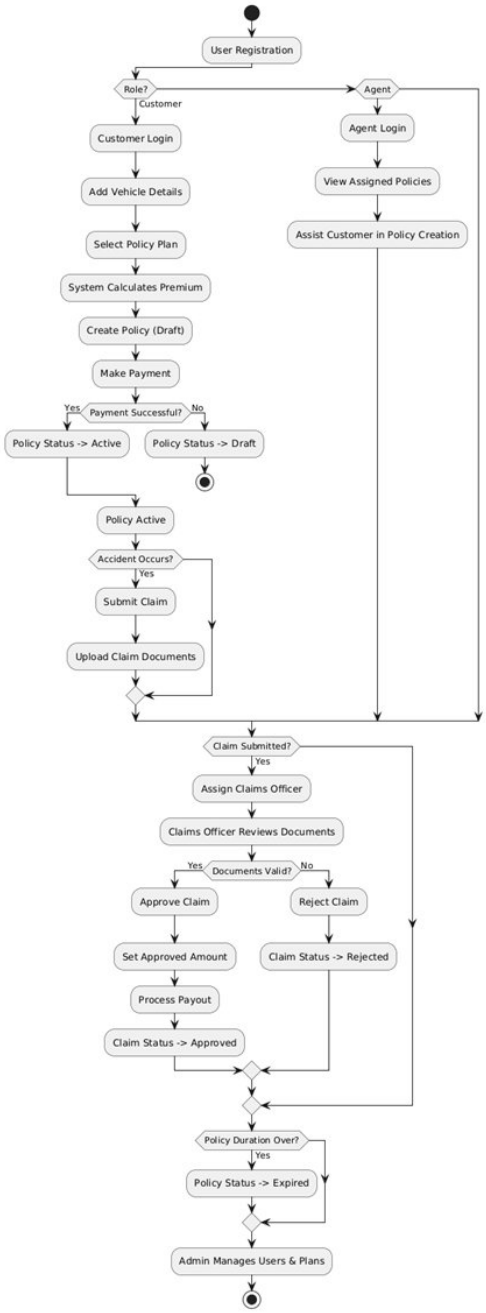

Figure 4: Activity Diagram — VIMS End-to-End Business Process Flow

9.5. Policy Sequence Diagram

The Policy Purchase Sequence Diagram shows the time-ordered message flow between the Customer, Insurance System, and Payment Gateway during policy acquisition. It illustrates the two alternate paths — Payment Successful (Policy Activated, Confirmation sent) and Payment Failed (Policy remains Draft, failure message returned) — using a UML alt fragment.

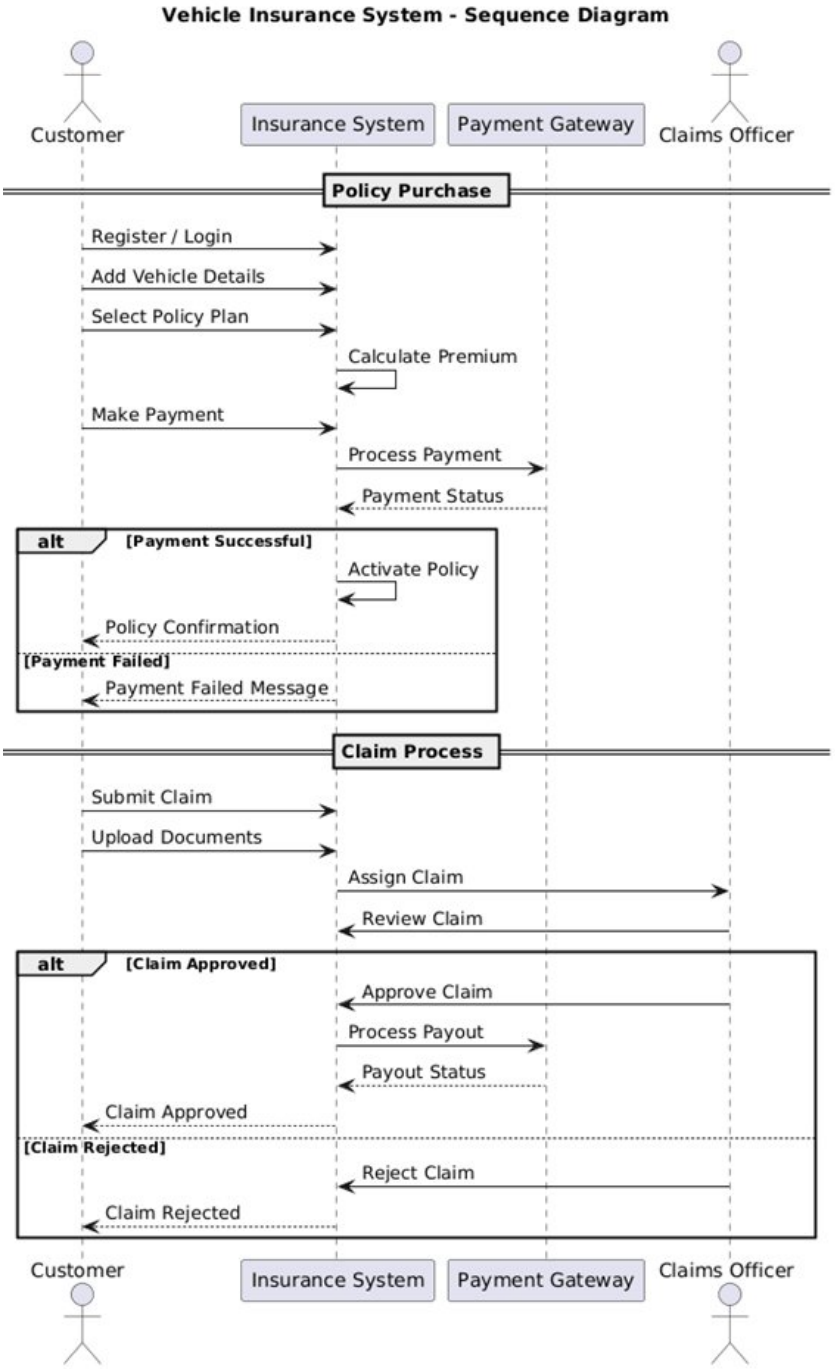

Figure 5: Sequence Diagram — Policy Purchase and Claim Process

10. Database Design

The VehicleInsuranceDB database runs on Microsoft SQL Server and is managed through Entity Framework Core Code-First migrations. The schema is normalized to Third Normal Form (3NF) to eliminate data redundancy and enforce referential integrity. All financial fields use the DECIMAL type to prevent floating-point rounding errors.

10.1. Major Entities and Relationships

Entity	Primary Key	Key Columns	Purpose
Users	UserId	Email (UNIQUE), PasswordHash, Role (enum: 0-3), IsActive, SecurityQuestion, SecurityAnswerHash	All system users — Customers, Agents, Claims Officers, Admins
PolicyPlans	PlanId	PlanName, PolicyType, BasePremium, MaxCoverageAmount, DeductibleAmount, CoverageFlags (OwnDamage/Theft/ThirdParty), AddOnFlags, Status	Insurance plan templates available for customer purchase
VehicleApplications	ApplicationId	CustomerId (FK), AssignedAgentId (FK), PlanId (FK), RegistrationNumber, VehicleType, KilometersDriven, Status, InvoiceAmount	Customer applications for vehicle registration and policy issuance
VehicleDocuments	DocumentId	VehicleApplicationId (FK, CASCADE), DocumentType, FilePath	Supporting documents uploaded with a vehicle application
Vehicles	VehicleId	CustomerId (FK), VehicleApplicationId (FK, UNIQUE), RegistrationNumber (UNIQUE), Make, Model, Year, FuelType, CurrentMarketValue	Approved and registered vehicles in the system
Policies	PolicyId	PolicyNumber (UNIQUE), CustomerId (FK), AgentId (FK), VehicleId (FK), PlanId (FK), Status, PremiumAmount, IDV, InvoiceAmount, StartDate, EndDate, SelectedYears, CurrentYearNumber, CurrentYearEndDate, IsCurrentYearPaid, ClaimCount, IsRenewed	Active insurance contracts

Entity	Primary Key	Key Columns	Purpose
Claims	ClaimId	ClaimNumber (UNIQUE), PolicyId (FK), CustomerId (FK), ClaimsOfficerId (FK nullable), ClaimType (enum), Status, ApprovedAmount, DecisionType, RejectionReason, Document1, Document2	Insurance claim records and adjudication history
Payments	PaymentId	PolicyId (FK, CASCADE), Amount, PaymentDate, Status, PaymentMethod, TransactionReference (UNIQUE)	All financial transactions for policy premiums and claim payouts
PolicyTransfers	TransferId	PolicyId (FK), SenderCustomerId (FK), RecipientCustomerId (FK), Status, RejectionReason	Policy ownership transfer requests between customers
AuditLogs	AuditId	UserId (nullable FK), Email, Role, Action, Category, EntityName, EntityId, Details, Timestamp	Write-once immutable record of every critical system action

Relationship	Cardinality	Cascade Behaviour
Users → VehicleApplications (Customer)	1 : Many	None — applications persist if user is deactivated
VehicleApplications → VehicleDocuments	1 : Many	CASCADE DELETE — documents removed with application
VehicleApplications → Vehicles	1 : 1	None — Vehicle persists after approval
Vehicles → Policies	1 : Many	CASCADE DELETE
PolicyPlans → Policies	1 : Many	CASCADE DELETE
Policies → Claims	1 : Many	CASCADE DELETE — claims removed with policy
Policies → Payments	1 : Many	CASCADE DELETE
Policies → PolicyTransfers	1 : Many	None — transfers retained for audit trail
Users → AuditLogs	1 : Many	SetNull — logs retained even if user deleted

11. API Documentation Overview

The VIMS REST API is organized into five controller groups and exposes 39 endpoints. All responses are JSON. Protected endpoints require a valid JWT Bearer token: Authorization: Bearer <token>. The full interactive Swagger documentation is available at /swagger/index.html when the application is running.

Base URL: https://<host>/api

11.1. Authentication APIs (JWT Engine)

Method	Endpoint	Auth Required	Purpose
POST	/api/Auth/customer/register	None	Register customer. AuthService validates CAPTCHA before creating account.
POST	/api/Auth/login	None	Authenticate with email + password + CAPTCHA. Returns signed JWT with UserId, Email, Role.
POST	/api/Auth/admin/register	None	Register Admin account (restrict in production).
PUT	/api/Auth/change-password	Any role	Change password for the currently authenticated user.
GET	/api/Auth/forgot-password/security-question/{email}	None	Retrieve security question text for password reset.
POST	/api/Auth/forgot-password/reset	None	Reset password by answering the security question.
POST	/api/Auth/set-security-question	Any role	Set or update security question and hashed answer.

11.2. Policy Management APIs

Method	Endpoint	Role	Purpose
GET	/api/Customer/all-policy-plans	Customer	Get all active PolicyPlans.
POST	/api/Customer/calculate-quote	Customer	Instant premium estimate — no records created.
POST	/api/Customer/vehicle-application	Customer	Submit new vehicle application with documents.

Method	Endpoint	Role	Purpose
GET	/api/Customer/my-applications	Customer	Get own applications.
GET	/api/Customer/my-policies	Customer	Get own policies.
GET	/api/Customer/policy/{policyId}	Customer	Full policy details.
POST	/api/Customer/pay-annual/{policyId}	Customer	Pay annual premium.
POST	/api/Customer/renew/{policyId}	Customer	Renew policy for next year.
POST	/api/Customer/policy/cancel/{policyId}	Customer	Cancel an active policy.
POST	/api/Customer/transfer/initiate	Customer	Initiate policy transfer to another customer.
POST	/api/Customer/transfer/{id}/accept	Customer	Accept incoming transfer.
POST	/api/Customer/transfer/{id}/reject	Customer	Reject incoming transfer.
PUT	/api/Agent/vehicle-application/{id}/review	Agent	Approve or Reject application — triggers Vehicle + Policy creation on approval.
GET	/api/Agent/pending-applications	Agent	Pending applications assigned to this agent.
POST	/api/Admin/createPolicyPlan	Admin	Create new PolicyPlan.
GET	/api/Admin/policy-plans	Admin	All PolicyPlans (active + inactive).
PUT	/api/Admin/deactivate/{id}	Admin	Deactivate user account.
PUT	/api/Admin/activate/{id}	Admin	Activate user account.

11.3. Claim Management APIs

Method	Endpoint	Role	Purpose
POST	/api/Customer/claim/submit	Customer	Submit new claim — ClaimsService validates coverage.
GET	/api/Customer/claims/my	Customer	Get own claims.
GET	/api/Customer/claim/{id}	Customer	Specific claim details.
GET	/api/Customer/payments/my	Customer	Own payment records.
GET	/api/Customer/invoice/download/{paymentId}	Customer	Download PDF invoice.
GET	/api/ClaimsOfficer/all-claims	Claims Officer	All claims (system-wide queue).

Method	Endpoint	Role	Purpose
GET	/api/ClaimsOfficer/my-claims	Claims Officer	Claims assigned to this officer.
GET	/api/ClaimsOfficer/claim/{id}	Claims Officer	Full claim details with documents.
POST	/api/ClaimsOfficer/decide/{claimId}	Claims Officer	Approve/Reject decision — enforces payout cap.
GET	/api/Admin/claims	Admin	All claims system-wide.
GET	/api/Admin/payments	Admin	All payment records.
GET	/api/Admin/audit-logs	Admin	Immutable AuditLog entries.

12. Key Features & Technical Challenges

12.1. Key Features

Feature	Implementation	Business Value
JWT Security	AuthService issues signed JWTs on login after CAPTCHA validation. Every protected endpoint validates via ASP.NET Core middleware.	Stateless, scalable authentication. Tokens expire and cannot be forged without the signing key.
Flexible RBAC	Four roles enforced at backend (Authorization attributes) and frontend (Route Guards). Each endpoint declares its required role.	Users can only access data and operations strictly within their designated scope.
CAPTCHA Validation	AuthService validates CAPTCHA on every login and registration attempt before processing any credentials.	Prevents brute-force attacks, credential stuffing, and automated bot registrations.
Automated Invoicing	PDF Generator creates downloadable invoices with PolicyNumber, amount, date, PaymentMethod, and TransactionReference.	Customers receive formal financial documentation for every transaction automatically.
Immutable AuditLog	Every critical action is written to AuditLogs. No update or delete operations exist for this table at any layer.	Full accountability for compliance, dispute resolution, and fraud investigation.
Dynamic Pricing Engine	PricingService calculates IDV, premium, and InvoiceAmount server-side. Client-submitted price values are completely ignored.	Tamper-proof, auditable, and consistent pricing across all transactions.
Status State Machines	Policy and Claim statuses follow strict allowed transitions enforced by the service layer. Invalid transitions return HTTP 400.	Data integrity guaranteed — records cannot skip stages or enter invalid states.
Policy Transfer Workflow	Transfers require both sender initiation and recipient acceptance before any ownership change is applied.	Two-party consent prevents unauthorized transfers. Both parties must actively confirm.

12.2. Development Challenges & Solutions

Challenge	Solution Adopted
Complex Claim Payout Calculations: Payout logic varies by ClaimType and plan type (Standard vs. Zero Dep). Inconsistency creates financial exposure.	Centralized all logic in ClaimsService.CalculatePayout() using a switch expression on ClaimType. PricingService and ClaimsService share the same IDV utility — zero code duplication.
JWT Token Persistence: Stateless tokens must be stored client-side without XSS exposure.	Angular HTTP Interceptor reads token from localStorage on every request. On 401, clears token and redirects to login. Route Guards decode the token payload for role-based navigation control.
Large File Upload Organization: Thousands of uploaded documents without structure cause collisions and make retrieval difficult.	Structured directory hierarchy: wwwroot/uploads/{userId}/{category}/{timestamp_filename}. Database stores only relative file paths. Files served as static content; easily backed up without touching the DB.
Concurrent Claim Prevention: Customers could file duplicate claims for the same incident on the same policy.	ClaimsService validates no Pending or UnderReview claim exists for the same PolicyId + ClaimType before accepting a new submission. Partial unique constraint at database level provides secondary defence.
EF Core Cascade Delete Conflicts: SQL Server disallows multiple cascade paths to the same table.	Cascade deletes configured selectively via EF Core Fluent API. Only natural parent-child paths cascade. Secondary relationships use Restrict or SetNull — eliminating the multi-cascade-path SQL Server error.
PricingService Enforcement: Client could send a different InvoiceAmount to underpay premiums.	Endpoints completely ignore any price values in the request body. PricingService is always called server-side and its output used directly. The client only submits vehicle details and plan selection.

13. Future Enhancements

VIMS is architecturally designed for extensibility. The following enhancements represent natural evolutionary steps on the existing modular foundation — each achievable without structural changes to the core system:

- **VAHAN API Integration:** Automatically fetch vehicle registration details (make, model, registered owner) from the government VAHAN API using just the registration number — eliminating manual entry and reducing fraud risk. Implemented as a VahanService in the Application layer, called during VehicleApplication submission.
- **Online Payment Gateway:** Integrate Razorpay or Stripe for real-time premium payment processing with webhook-based confirmation. Razorpay order IDs stored alongside Payment records; a dedicated webhook controller updates Payment.Status on receipt of confirmation events.
- **AI Fraud Detection:** Deploy a machine learning classification model to flag suspicious claim patterns — unusually high claim frequencies, suspicious timing, or inflated repair estimates — before they reach the Claims Officer. Exposed as an internal microservice called by ClaimsService during submission.
- **Mobile Application:** A React Native or Flutter app allowing customers to file claims on-the-spot with integrated camera capture for damage photos, GPS-tagged incident location, and push notifications via Firebase Cloud Messaging. The existing REST API is fully reusable.
- **Advanced Analytics Dashboard:** A Business Intelligence dashboard for Admins showing claim ratios, premium revenue trends, plan performance, agent productivity metrics, and fraud rate indicators — built with Power BI Embedded or custom D3/Recharts visualizations.
- **Automated Email & SMS Notifications:** Notifications for application approval/rejection, claim status changes, policy renewal reminders, and payment receipts — delivered via SendGrid (email) and Twilio (SMS) through a background NotificationService using Hangfire or .NET hosted services.

14. Conclusion

The Vehicle Insurance Management System (VIMS) provides a scalable and robust digital foundation for modern vehicle insurance operations. By systematically replacing legacy manual processes with automated, role-based digital workflows, the system delivers measurable improvements in operational efficiency, financial precision, customer transparency, and regulatory compliance.

The system's N-Tier layered architecture — ASP.NET Core Web API with a Service-Repository pattern, Angular 20+ SPA with HTTP Interceptors and Route Guards, and EF Core Code-First with SQL Server — ensures that each concern is handled in the right place by the right component. The PricingService enforces consistent, tamper-proof financial calculations. The ClaimsService manages the complete claim lifecycle with proper coverage validation and payout caps. The immutable AuditLog provides an unalterable record of every system action for compliance and fraud investigation.

The six future enhancements outlined — VAHAN API integration, online payment gateways, AI fraud detection, mobile applications, analytics dashboards, and automated notifications — are all achievable incremental additions that build on this solid foundation without requiring architectural restructuring. This forward-compatible design makes VIMS a genuinely valuable, production-ready asset for any insurance enterprise looking to modernize its operations and deliver a superior digital experience to its customers.

— End of Document —